AI-Powered RIS Phase Shift Optimization for 6G Networks

Project Report

Arda Özgür — B.Sc. Computer Engineering
Mudanya University

1. Abstract

Reconfigurable Intelligent Surfaces (RIS) are passive arrays of reflecting elements whose phase shifts can be tuned to steer signals toward a user, and they are a leading candidate technology for 6G. Choosing the phase shifts is hard: the objective is non-convex, and classical iterative solvers such as Alternating Optimization (AO) must be re-run every time the channel changes. This project trains a small neural network that maps the channel state directly to RIS phase shifts in a single forward pass. The network is trained without labels by back-propagating through the spectral-efficiency formula itself. On a simulated MISO downlink with M = 4 base-station antennas and N = 16 RIS elements, the network reaches 11.25 bps/Hz against 11.61 bps/Hz for the AO reference, about 97% of it, while requiring only one inference step instead of an iterative search.

2. Background and Motivation

In an RIS-assisted MISO system, the user receives the direct signal from the base station (BS) plus a signal reflected through the RIS. With N phase shifts to choose, the effective channel is

$$h_{eff} = h_d^H + h_r^H \cdot \Theta \cdot G, \quad \Theta = \mathrm{diag}(e^{j\theta_1}, \ldots, e^{j\theta_N})$$

where h_d is the BS-to-user channel (M×1), G is the BS-to-RIS channel (N×M) and h_r is the RIS-to-user channel (N×1). With maximum-ratio transmission (MRT) at the BS, the received SNR is $P_t \cdot \|h_{eff}\|^2 / \sigma^2$, and the objective is the spectral efficiency $\log_2(1 + \mathrm{SNR})$.

Iterative solvers handle this well but are slow, and their cost grows with the number of elements. Wireless channels change within milliseconds, so a solver that is too slow is useless in practice. The idea here is to learn the mapping from channel to phases once, offline, and then evaluate it cheaply online.

3. System Model and Simulation Setup

- **Geometry.** BS at (0, 0, 10) m, RIS at (50, 10, 10) m, user at (60, 0, 1.5) m.
- **Path loss.** Reference loss of −30 dB at 1 m; exponent 3.5 for the direct path and 2.2 for the two RIS links, which are more likely to have line of sight.
- **Small-scale fading.** The direct link uses Rayleigh fading (no line of sight). Both RIS links use Rician fading with K = 3, built from a deterministic all-ones LoS term plus a Rayleigh scattered term.
- **Link budget.** Transmit power 30 dBm (1 W), noise power −90 dBm.
- **Dimensions.** M = 4 BS antennas, N = 16 RIS elements, one single-antenna user.

4. Software Architecture

File	Role
channel_model.py	Path-loss model and channel generator producing (h_d, G, h_r) for each realization
baselines.py	Reference methods: No-RIS, Random Phase, and Alternating Optimization
environment.py	Gymnasium environment (single-step episodes, reward = spectral efficiency) so RL agents such as DDPG, TD3, PPO or SAC can be plugged in
train.py	Network definition, differentiable loss, training loop and evaluation
plots_and_benchmarks.py	Power sweep, 2-bit quantization test, latency measurement and plotting
ris_model.pth	Saved weights of the trained network

5. Methodology

5.1 Neural network

RISPhaseNet is a fully connected network. The input is the flattened real and imaginary parts of h_d, G and h_r, which gives $2(M + NM + N) = 168$ values, scaled by 10^3 to counter the tiny magnitudes caused by path loss. The hidden layers are 256 and 128 units, each followed by BatchNorm and ReLU. The output layer has N units with a sigmoid, and the result is multiplied by 2π so every phase lies in $[0, 2\pi]$.

5.2 Unsupervised, differentiable loss

Instead of using AO solutions as labels, the training loss is written entirely in PyTorch: the predicted phases are converted to complex exponentials, the effective channel is computed with batched matrix products, and the loss is the negative mean spectral efficiency. Gradients therefore flow from the rate directly back to the network weights. No labelled data is needed, so the network is not limited by the quality of a teacher algorithm.

5.3 Training

Adam optimizer, learning rate 1e-3, batch size 64, 200 iterations. A fresh batch of channels is sampled at every iteration, so the model never sees the same channel twice.

5.4 Baselines

- **No RIS:** only the direct path.
- **Random phase:** phases drawn uniformly from $[0, 2\pi)$.
- **Alternating Optimization:** coordinate ascent. Each element in turn is set to the phase that aligns its reflected contribution with the sum of all the others, repeated for 20 sweeps. For this single-user MRT problem each coordinate update is exact.

5.5 Hardware-constrained variant

The benchmark script also rounds the predicted phases to a 2-bit grid (four levels, 90° apart) to model practical metasurfaces with discrete phase control.

6. Results

6.1 Spectral efficiency comparison

Averaged over 100 test channels at 30 dBm:

Method	Spectral efficiency (bps/Hz)	Gain over No RIS	Share of AO
No RIS	11.04	–	95.1%
Random phase	11.06	+0.02	95.3%
AI-powered RIS (this work)	11.25	+0.21	96.9%
Alternating Optimization	11.61	+0.57	100%

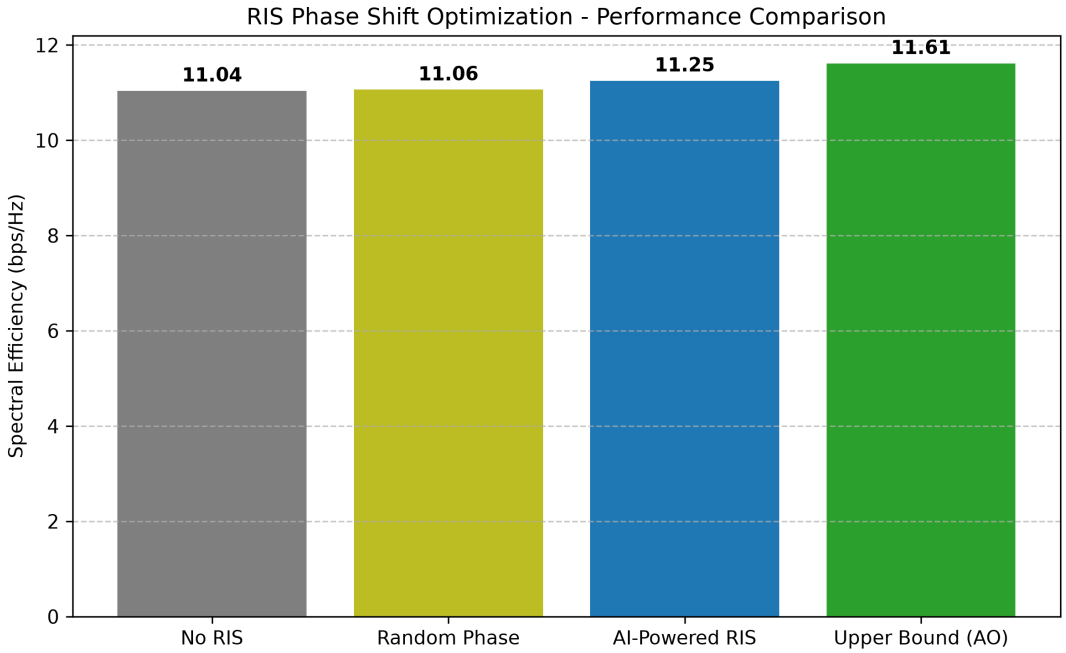


Figure 1. Average spectral efficiency of the four methods (benchmark_results.png).

The ordering is as expected: random phases give essentially no benefit over having no RIS, because random reflections add up incoherently. The trained network clearly beats random phases and recovers roughly 37% of the gain that AO achieves over No RIS (0.21 of 0.57 bps/Hz).

6.2 Latency

The benchmark script times AO and network inference per channel. The README reports 28.45 ms for AO against 0.32 ms on CPU (about 89×) and 0.08 ms on GPU (about 350×), and a speedup of up to 591× at N = 128. The qualitative conclusion is sound, since a single forward pass of a small network is much cheaper than 20 sweeps of per-element matrix operations, but see Section 7 on how these figures should be presented.

7. Discussion and Honest Assessment

What works well. The label-free differentiable training is a clean design, the channel model is more realistic than the i.i.d. Rayleigh models common in coursework, and the project is well organised, with a Gym-compatible environment ready for reinforcement-learning experiments.

Points to tighten before presenting the project.

1. **README numbers do not match the figure.** The README states about 10.9 bps/Hz for AO and 10.8 for the network, while the saved benchmark shows 11.61 and 11.25. The figure is the more trustworthy source; regenerate the table from a single run so everything agrees.
2. **Absolute rates are dominated by the direct link.** At 30 dBm the direct path alone already gives about 34 dB SNR, so the rate is 11 bps/Hz before any RIS is used. Because of the logarithm, a large improvement in channel gain shows up as a small improvement in bps/Hz. Reporting gains in dB of channel power, or running a lower-power or weaker-direct-link scenario, would show the RIS benefit more clearly.
3. **AO is called a theoretical upper bound.** It is a coordinate-ascent heuristic and can stop at a local optimum, so it is a strong reference rather than a proven bound. Also, the code comment says the initial phases are random but they are initialised to zero.
4. **Training is short.** 200 iterations of 64 samples is quick, but the gap to AO (about 3%) suggests the network has not converged. Longer training, a learning-rate schedule or a wider network would probably close part of it. The loop prints no validation metric, so you cannot tell whether it has plateaued.
5. **Features that are claimed but only partly implemented.** The scripts quantize to 2 bits only; there is no 1-bit run. The N = 128 scaling, the 591× figure and the separate GPU timing are not produced by the supplied code, since the network is built for N = 16 and the benchmark fixes M = 4, N = 16. If these results came from other scripts, include them in the repository.
6. **Timing method.** AI timing covers one forward pass for a single channel, while AO runs in NumPy. Comparing different hardware or batching setups can overstate the speedup, so describe exactly what is measured and on which hardware.
7. **Single geometry.** The positions are constants, so the model learns only the small-scale fading statistics. Generalising across user positions would be a much stronger result.
8. **README clean-up.** Remove the leftover "[cite: ...]" markers, and make the Coursera credential link point directly to the certificate instead of a Google search URL.

8. Conclusion

The project shows that a compact neural network, trained through a differentiable spectral-efficiency loss with no labels, can configure a 16-element RIS in one forward pass and reach about 97% of the rate of an iterative Alternating Optimization baseline in this simulation, well above random phases and the no-RIS case. The remaining work is mostly about rigor: making the reported numbers consistent, measuring latency carefully, and testing in settings where the RIS matters more.

9. Future Work

- Train longer with a validation set and a learning-rate schedule to narrow the gap to AO.
- Add 1-bit quantization and quantization-aware training (for example a straight-through estimator) so the network learns to output discrete phases.
- Randomise user and RIS positions to test generalisation across geometries.
- Add a lower-power or blocked-direct-link scenario where the RIS dominates.
- Use the Gymnasium environment to compare against RL agents (TD3, SAC, PPO).

- Scale to N = 64 and 128 with an architecture suited to larger arrays, and measure latency properly on CPU and GPU.
- Extend to multi-user MISO with sum-rate objectives and imperfect channel estimates.

10. How to Reproduce

```
pip install -r requirements.txt
python train.py
python plots_and_benchmarks.py
```

Dependencies: numpy ≥ 1.22, torch ≥ 2.0, gymnasium ≥ 0.28, matplotlib ≥ 3.5.

11. Acknowledgement

This project was developed alongside insights from the University of Glasgow program “6G Vision: ML, Intelligent Surfaces & Optical Networks”.